

Task 4

WEB APPLICATION SECURITY ASSESSMENT REPORT

Security Assessment of DVWA (Damn Vulnerable Web Application)

1. INTRODUCTION

Web applications play a critical role in modern organizations by providing services and functionality through internet-based platforms. Due to their accessibility and complexity, web applications are often targeted by cyber attackers seeking unauthorized access to sensitive information.

Security assessments help identify vulnerabilities before they can be exploited by malicious actors. A vulnerability assessment involves systematically examining an application to discover weaknesses, evaluate associated risks, and recommend corrective actions.

DVWA provides a safe environment for learning and understanding real-world web application vulnerabilities. This project focuses on assessing several common vulnerabilities and documenting their risks and mitigation strategies.

2. OBJECTIVES

The primary objectives of this assessment were:

- To install and configure DVWA in a local testing environment.
- To identify common web application vulnerabilities.
- To perform vulnerability testing in an authorized environment.
- To understand the impact of discovered vulnerabilities.
- To document findings and remediation measures.
- To prepare a professional penetration testing report.

3. SCOPE OF ASSESSMENT

Target Application: DVWA (Damn Vulnerable Web Application)

Environment: Localhost

Testing Method: Manual Security Testing

Security Level Tested: Low

Verification Level: Impossible

Assessment Type: Vulnerability Assessment and Security Testing

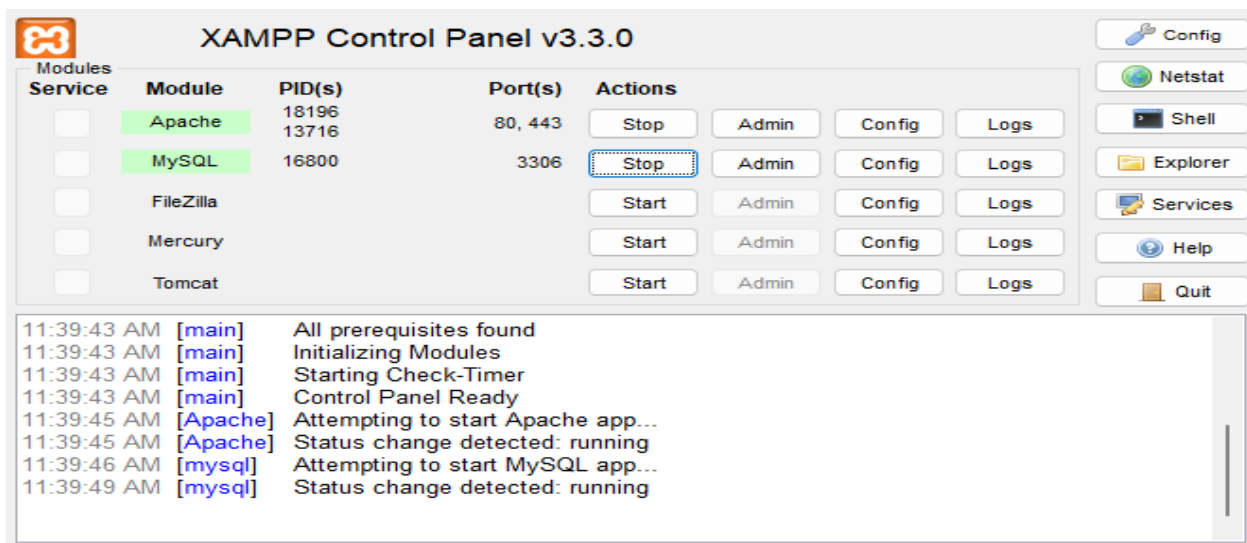
4. TOOLS USED

The following tools were utilized during the assessment:

1. XAMPP
 - Used for hosting the DVWA application locally.
2. DVWA
 - Vulnerable web application used as the testing target.
3. Burp Suite Community Edition
 - Used for intercepting and analyzing HTTP requests.
4. Web Browser
 - Used to access and test the application.

5. TEST ENVIRONMENT SETUP

The DVWA application was successfully installed and configured on a local machine using XAMPP. Apache and MySQL services were started, and the DVWA database was initialized successfully.





Username

Password



- Home
- Instructions
- Setup / Reset DB
- Brute Force
- Command Injection
- CSRF
- File Inclusion
- File Upload
- Insecure CAPTCHA
- SQL Injection
- SQL Injection (Blind)
- Weak Session IDs
- XSS (DOM)
- XSS (Reflected)
- XSS (Stored)
- CSP Bypass
- JavaScript Attacks
- Authorisation Bypass
- Open HTTP Redirect

Welcome to Damn Vulnerable Web Application!

Damn Vulnerable Web Application (DVWA) is a PHP/MySQL web application that is damn vulnerable. Its main goal is to be an aid for security professionals to test their skills and tools in a legal environment, help web developers better understand the processes of securing web applications and to aid both students & teachers to learn about web application security in a controlled class room environment.

The aim of DVWA is to **practice some of the most common web vulnerabilities**, with various levels of difficulty, with a simple straightforward interface.

General Instructions

It is up to the user how they approach DVWA. Either by working through every module at a fixed level, or selecting any module and working up to reach the highest level they can before moving onto the next one. There is not a fixed object to complete a module; however users should feel that they have successfully exploited the system as best as they possible could by using that particular vulnerability.

Please note, there are **both documented and undocumented vulnerabilities** with this software. This is intentional. You are encouraged to try and discover as many issues as possible.

There is a help button at the bottom of each page, which allows you to view hints & tips for that vulnerability. There are also additional links for further background reading, which relates to that security issue.

WARNING!

Damn Vulnerable Web Application is damn vulnerable! Do not upload it to your hosting provider's public html folder or any Internet facing servers, as they will be compromised. It is recommend using a virtual machine (such as [VirtualBox](#) or [VMware](#)), which is set to NAT networking mode. Inside a guest machine, you can download and install [XAMPP](#) for the web server and database.

Disclaimer

6. VULNERABILITY FINDINGS

6.1 SQL INJECTION

Description

SQL Injection is a vulnerability that occurs when user-supplied input is incorporated directly into SQL queries without proper validation. Attackers can manipulate database queries to retrieve, modify, or delete data.

Testing Procedure

The SQL Injection module of DVWA was accessed and a crafted SQL payload was entered into the User ID field.

Payload Used

1' OR '1'='1

Observation

The application returned multiple database records, confirming that the input was executed as part of the SQL query.

DVWA

Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect

Vulnerability: SQL Injection

User ID:

ID: 1' OR '1'='1
First name: admin
Surname: admin

ID: 1' OR '1'='1
First name: Gordon
Surname: Brown

ID: 1' OR '1'='1
First name: Hack
Surname: Me

ID: 1' OR '1'='1
First name: Pablo
Surname: Picasso

ID: 1' OR '1'='1
First name: Bob
Surname: Smith

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

Impact

- Authentication bypass
- Unauthorized database access
- Sensitive data disclosure
- Potential database compromise

Risk Level

Critical

Recommended Mitigation

- Use parameterized queries.

- Implement prepared statements.
- Validate and sanitize user inputs.
- Apply least privilege access controls.

6.2 CROSS-SITE SCRIPTING (XSS)

Description

Cross-Site Scripting (XSS) occurs when user-controlled input is reflected back to the browser without proper encoding, allowing attackers to execute malicious scripts.

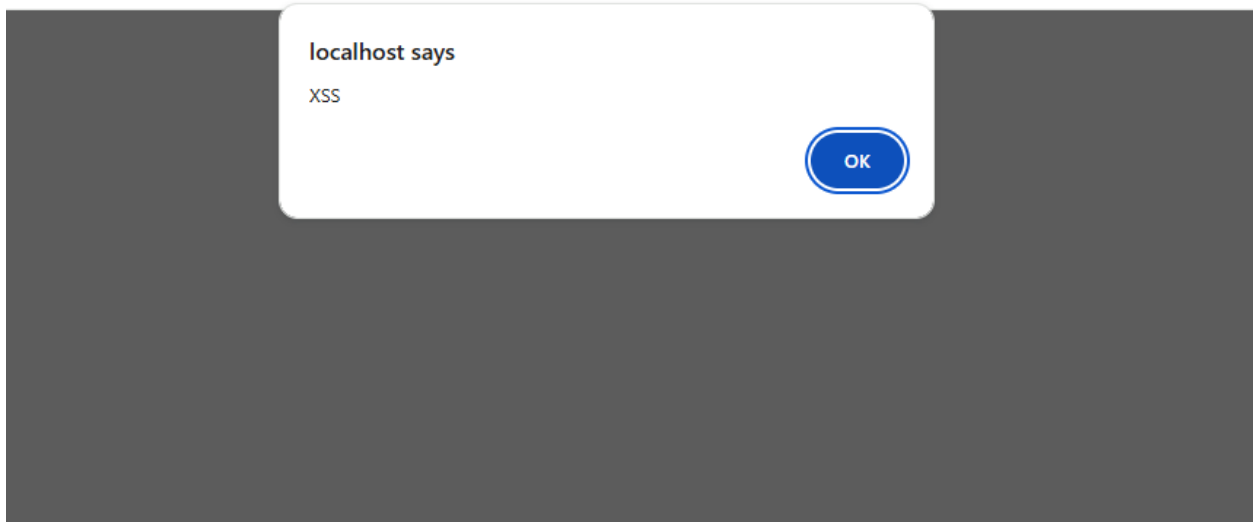
Testing Procedure

A JavaScript payload was entered into the XSS Reflected module.

Payload Used

Observation

The browser executed the JavaScript code and displayed an alert box.



Impact

- Session hijacking
- Cookie theft
- User impersonation
- Client-side attacks

Risk Level

High

Recommended Mitigation

- Encode output properly.
- Validate all user input.
- Implement Content Security Policy (CSP).
- Use secure cookie attributes.

6.3 BRUTE FORCE ATTACK

Description

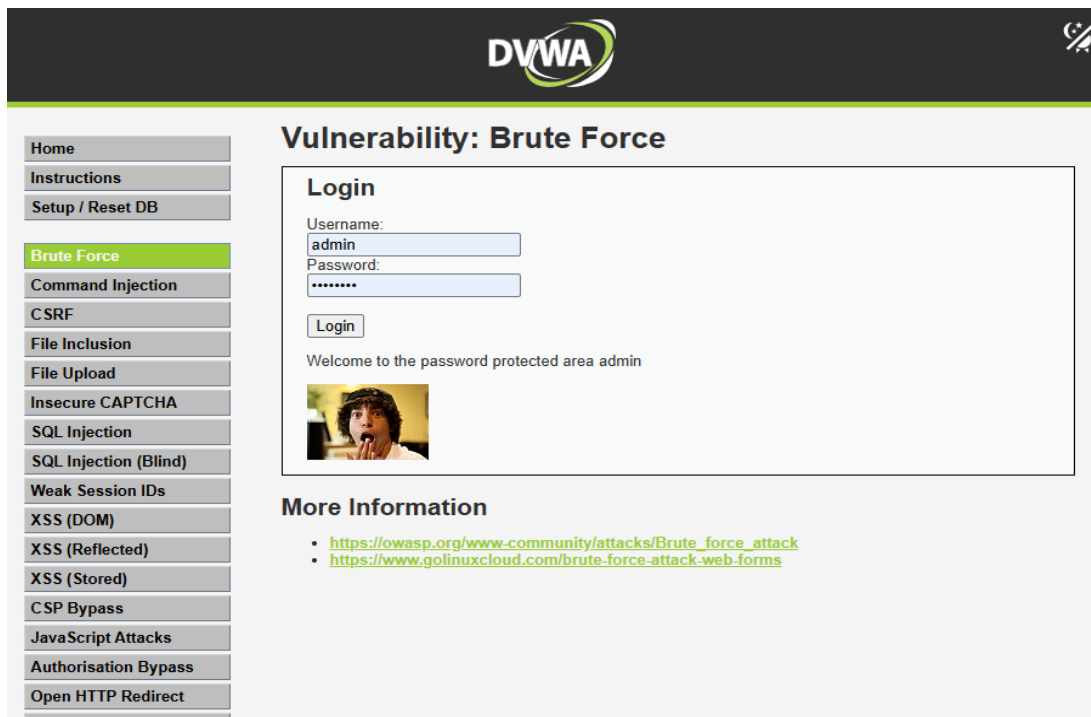
Brute Force attacks attempt to gain unauthorized access by repeatedly guessing usernames and passwords.

Testing Procedure

Multiple login attempts were performed using common credential combinations.

Observation

The application allowed unlimited authentication attempts without account lockout or rate limiting.



The screenshot displays the DVWA (Damn Vulnerable Web Application) interface. The top navigation bar includes the DVWA logo and a user profile icon. The left sidebar contains a list of vulnerability categories, with 'Brute Force' highlighted in green. The main content area is titled 'Vulnerability: Brute Force' and features a 'Login' form. The form has fields for 'Username' (containing 'admin') and 'Password' (containing '*****'), followed by a 'Login' button. Below the form, a message reads 'Welcome to the password protected area admin' next to a small image of a person with a surprised expression. At the bottom, a 'More Information' section lists two links: https://owasp.org/www-community/attacks/Brute_force_attack and <https://www.golinuxcloud.com/brute-force-attack-web-forms>.

Impact

- Unauthorized account access
- Credential compromise
- Increased attack surface

Risk Level

Medium

Recommended Mitigation

- Implement account lockout mechanisms.
- Enable multi-factor authentication.
- Apply rate limiting controls.
- Use CAPTCHA protections.

6.4 CROSS-SITE REQUEST FORGERY (CSRF)

Description

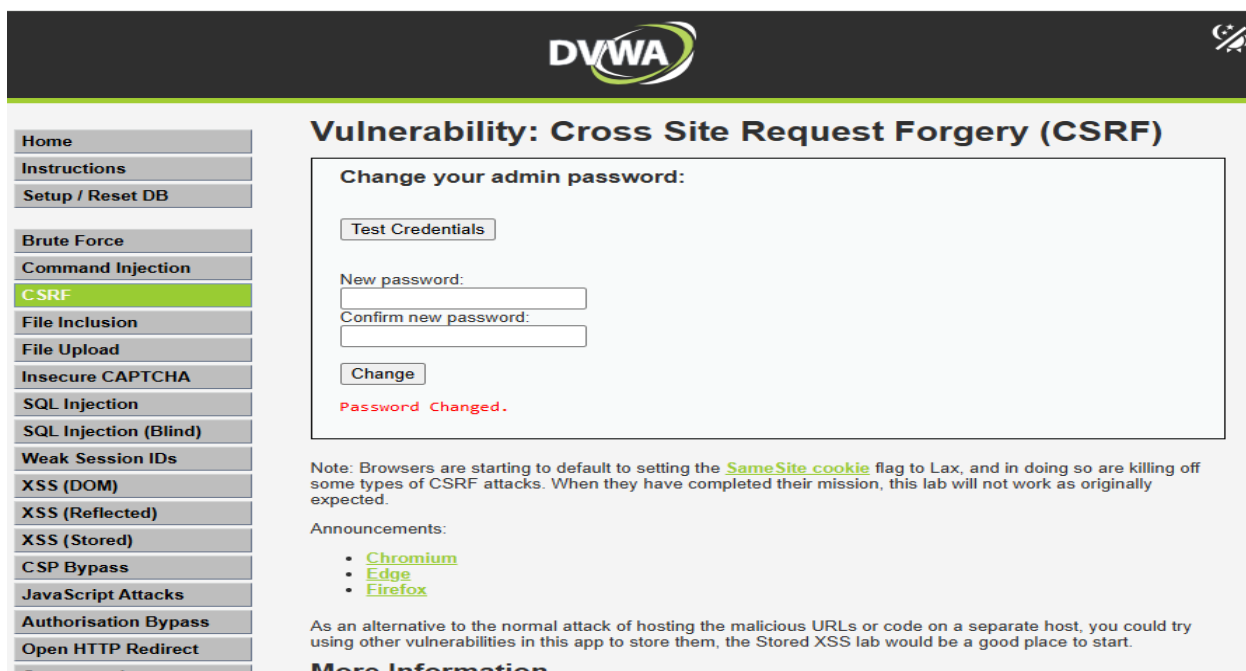
CSRF attacks trick authenticated users into performing unintended actions on a web application.

Testing Procedure

The password change functionality was examined for CSRF protection mechanisms.

Observation

Password changes could be initiated without verification of anti-CSRF tokens.



The screenshot shows the DVWA (Damn Vulnerable Web Application) interface. The top header features the DVWA logo and a small icon. The left sidebar contains a list of navigation links: Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, **CSRF** (highlighted in green), File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection, SQL Injection (Blind), Weak Session IDs, XSS (DOM), XSS (Reflected), XSS (Stored), CSP Bypass, JavaScript Attacks, Authorisation Bypass, and Open HTTP Redirect. The main content area is titled "Vulnerability: Cross Site Request Forgery (CSRF)". It contains a form titled "Change your admin password:" with a "Test Credentials" button, input fields for "New password:" and "Confirm new password:", and a "Change" button. Below the form, a red message states "Password Changed.". A note explains that browsers are starting to default to setting the SameSite cookie flag to Lax, which may affect the lab's functionality. A list of supported browsers (Chromium, Edge, Firefox) is provided under the "Announcements:" section. At the bottom, a note suggests using other vulnerabilities in the app to store malicious URLs or code on a separate host, and a "More Information" link is visible.

Impact

- Unauthorized actions
- Account manipulation
- Data modification

Risk Level

Medium

Recommended Mitigation

- Use anti-CSRF tokens.
- Implement SameSite cookie attributes.
- Validate request origins.

6.5 FILE INCLUSION

Description

File Inclusion vulnerabilities occur when user-controlled input is used to include files without proper validation.

Testing Procedure

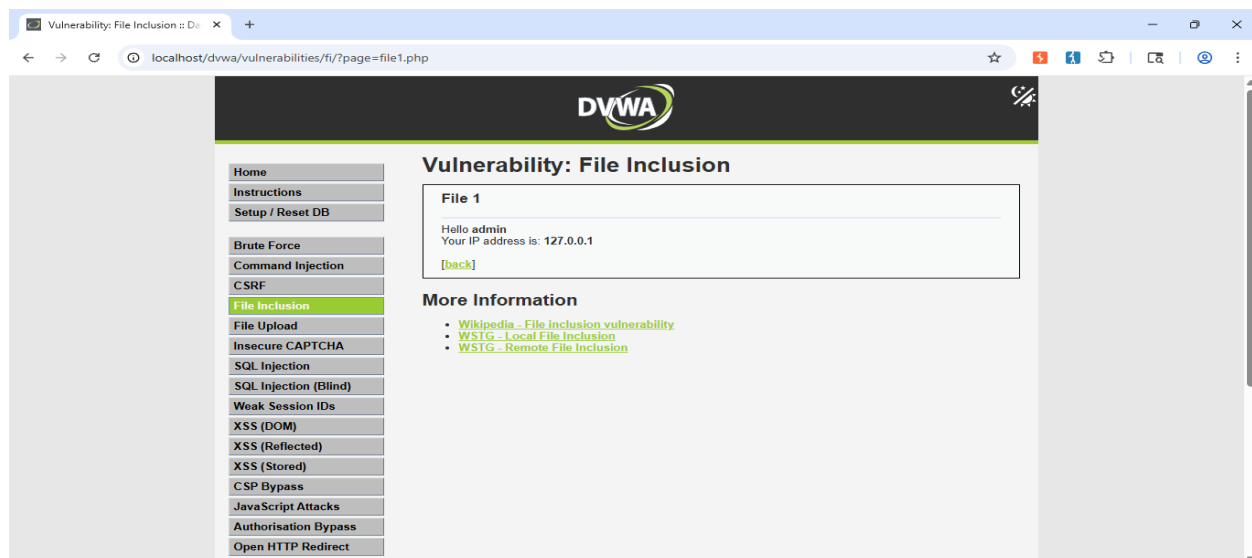
File path manipulation techniques were used to access local system files.

Payload Example

?page=../../../../../../windows/win.ini

Observation

The application attempted to access files outside the intended directory.



Impact

- Information disclosure
- Access to sensitive files
- Potential remote code execution

Risk Level

High

Recommended Mitigation

- Validate file paths.
- Use whitelisting mechanisms.
- Restrict file inclusion functionality.
- Disable unnecessary inclusion features.

7. RISK SUMMARY

Vulnerability	Severity
SQL Injection	Critical
Cross-Site Scripting (XSS)	High
File Inclusion	High
Brute Force Attack	Medium
Cross-Site Request Forgery (CSRF)	Medium

8. RECOMMENDATIONS

Based on the assessment findings, the following recommendations are proposed:

1. Implement secure coding standards across the application.
2. Use prepared statements for database interactions.
3. Validate and sanitize all user input.
4. Encode output before rendering in browsers.
5. Enable anti-CSRF protections.
6. Implement rate limiting and account lockout policies.
7. Apply the principle of least privilege.
8. Conduct periodic security assessments.

9. Keep software components updated.
10. Follow OWASP Top 10 security guidelines.

9. CONCLUSION

The Web Application Security Assessment successfully identified several critical and high-risk vulnerabilities within DVWA. These vulnerabilities demonstrated how insecure coding practices can lead to unauthorized access, information disclosure, and compromise of application functionality.

The assessment highlighted the importance of implementing secure development practices such as input validation, output encoding, parameterized database queries, anti-CSRF mechanisms, and authentication controls.

By applying the recommended mitigations and verifying protections using DVWA's Impossible security level, the application's security posture was significantly improved.

This assessment was conducted solely within an authorized local testing environment for educational and cybersecurity learning purposes.

REFERENCES

1. OWASP Top 10 Web Application Security Risks
2. DVWA Official Documentation
3. Burp Suite Community Edition Documentation
4. OWASP Testing Guide
5. Secure Coding Best Practices